

SQL JOIN = Building a Train

The Easiest Explanation You Will Ever Read

Think of a SQL query as building a train. You start with one locomotive. Then you attach carriages one by one. Each carriage is a table. The whole train is the virtual table (the combined result of all JOINS).

The most important thing to understand: once carriages are connected, they are part of one train. When you need something from the "d" carriage, you do NOT go to a separate "departments train". You look inside this train. The "d" is just the label on the 2nd carriage of THIS train.

Train Concept	SQL Concept	Real Meaning
Locomotive	FROM table	The starting table (base)
Carriage	Joined table	A table attached via JOIN
Coupling	ON condition	How two carriages connect
The whole train	Virtual table	Result of all JOINS combined
Carriage label (e, d, j...)	Table alias	A name tag on a carriage IN the train
Empty carriage	NULL values	No data for this carriage
Removing a carriage	INNER JOIN	If no match, whole train segment is cut off

Keep this mapping in mind as we go through each step.

Building the Train Step by Step

Step 1: The Locomotive

The query starts with FROM employees e. This is the locomotive - the starting point of our train. It has 5 compartments (columns): first_name, last_name, salary, job_id, department_id. The label on this locomotive is "e".

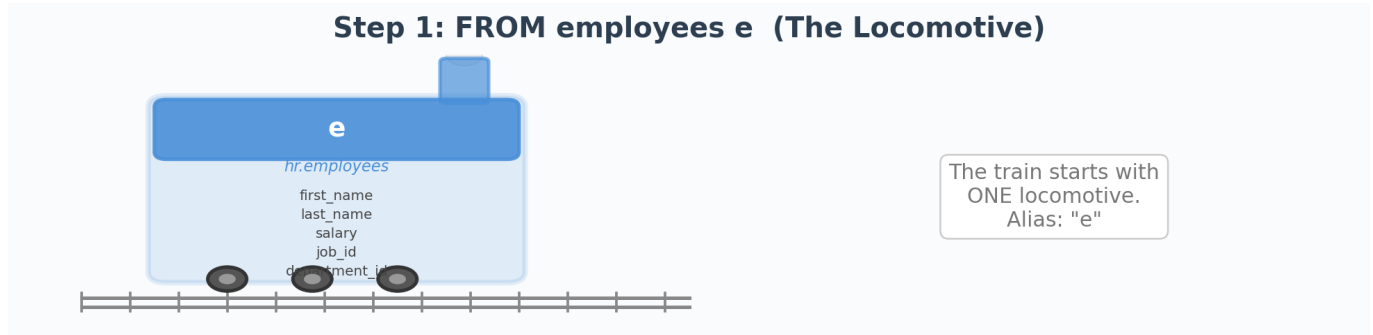


Figure 1: The locomotive. Alias "e". The whole train IS just this one car.

Step 2: Attach the Departments Carriage

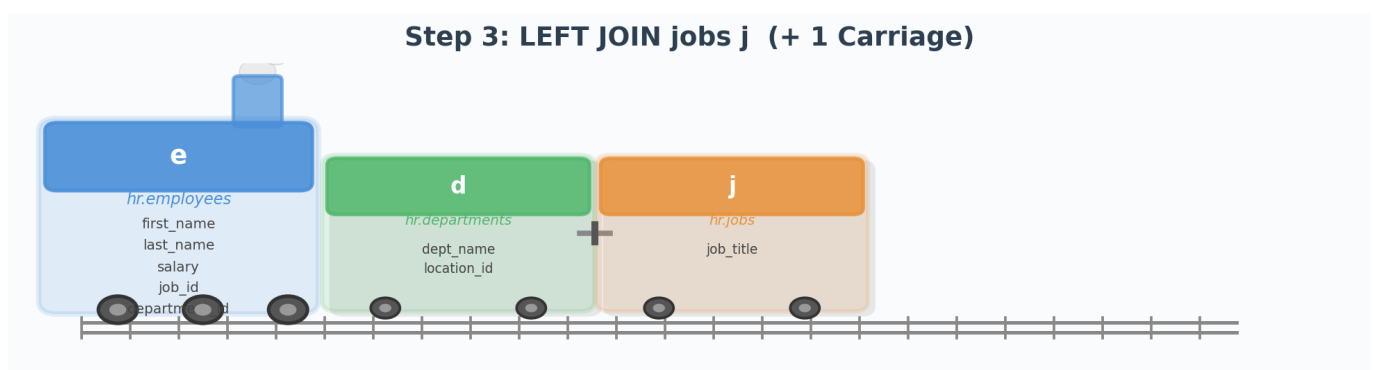
LEFT JOIN departments d ON e.department_id = d.department_id - This attaches a new carriage labeled "d". The coupling (ON clause) connects them by matching department_id. Now the train has 2 cars. They are ONE train. The "e" label is on the 1st car, "d" is on the 2nd car. There is no separate "e train" or "d train" anymore.



Figure 2: "d" carriage attached. The train now has e + d = 7 columns.

Step 3: Attach the Jobs Carriage

LEFT JOIN jobs j ON e.job_id = j.job_id - When SQL reads "e.job_id", it looks at the 1st car of THIS train (the locomotive labeled "e"). It finds job_id there, matches it with the jobs table, and attaches a new "j" carriage. The train now has 3 cars.



The Key Step: d.location_id

LEFT JOIN locations l ON d.location_id = l.location_id

This is the moment of confusion. People ask: "Where does d.location_id come from? We are not on the departments table anymore!"

Look at the train. The train currently has 3 cars: e, d, j. The "d" car (2nd carriage) has a compartment called location_id. SQL walks to the "d" carriage inside this train and reads location_id from there. It does NOT go to a separate "departments train". The departments data has been traveling with this train since Step 2.

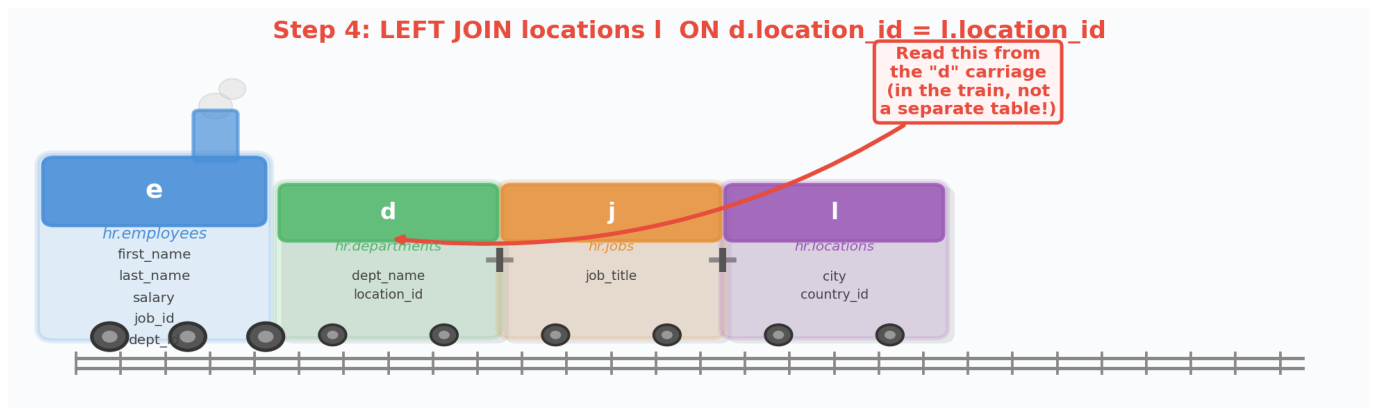


Figure 4: SQL reads d.location_id from the 2nd carriage of THIS train. Not from a separate table.

Think of it this way: You are in carriage "e" (the locomotive). You need to check something in carriage "d". Do you get off the train and go find another train called "departments"? No! You just walk to the 2nd carriage of your own train. That is where "d" is. It has been attached since Step 2.

The Complete Train

After all 6 JOINS, the train has 6 cars (including the locomotive). All data is in this one train. Every alias (e, d, j, l, c, r) is just a label on one of these cars.

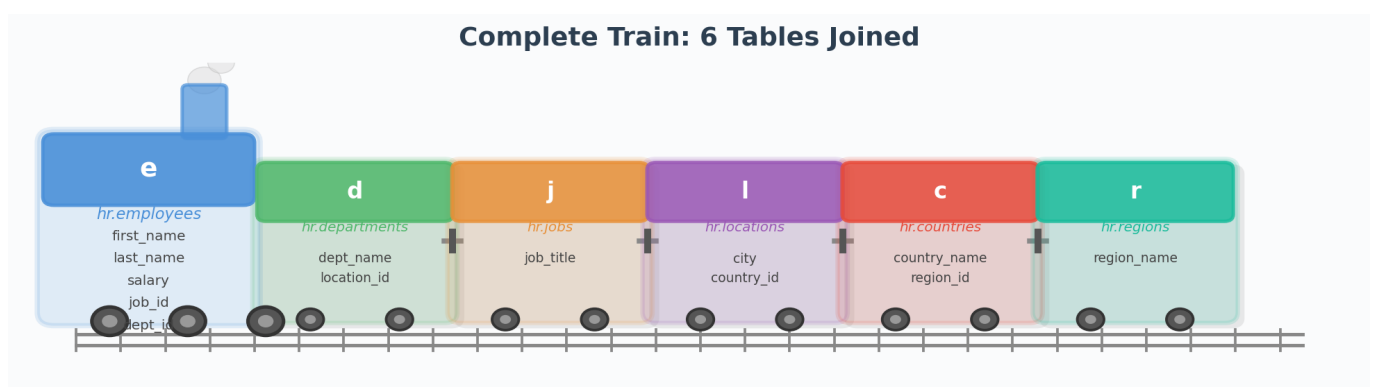


Figure 5: The complete train. 6 cars. ONE virtual table. Every alias is a carriage label.

LEFT JOIN vs INNER JOIN: What Happens to the Train?

Remember: we have 2 employees. Alice has a department (dept_id = 60). Kimberly has NO department (dept_id = NULL). How does this affect their trains?

With LEFT JOIN:

Both employees get their own train. Alice's train has all carriages full. Kimberly's "d" carriage is empty (NULL department). Because of the empty "d", the "l", "c", and "r" carriages are also empty (NULL cascade). But ALL carriages stay attached. The train is never shortened.

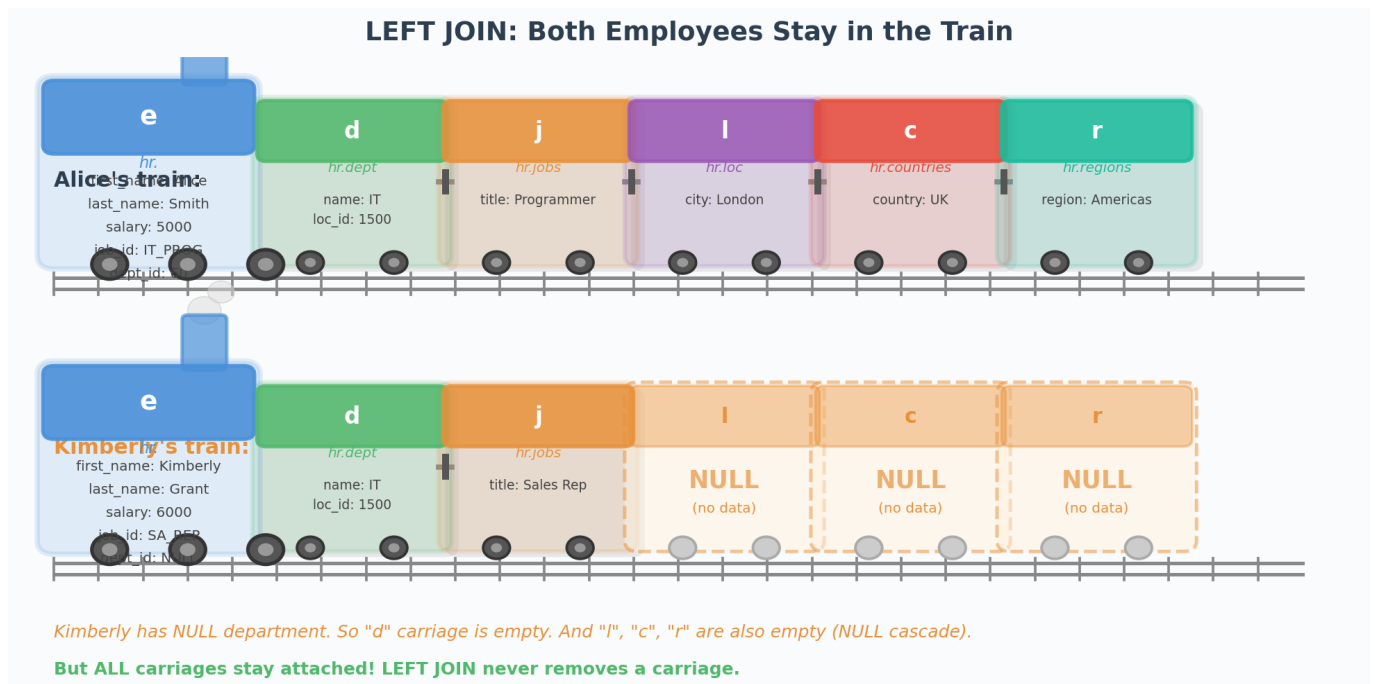


Figure 6: LEFT JOIN = keep all carriages, even if empty. Orange = NULL (no data).

With INNER JOIN:

INNER JOIN removes the ENTIRE train if any coupling fails. Kimberly has no department, so the coupling between "e" and "d" fails. Her ENTIRE train is removed. Not just the "d" carriage - the whole train.

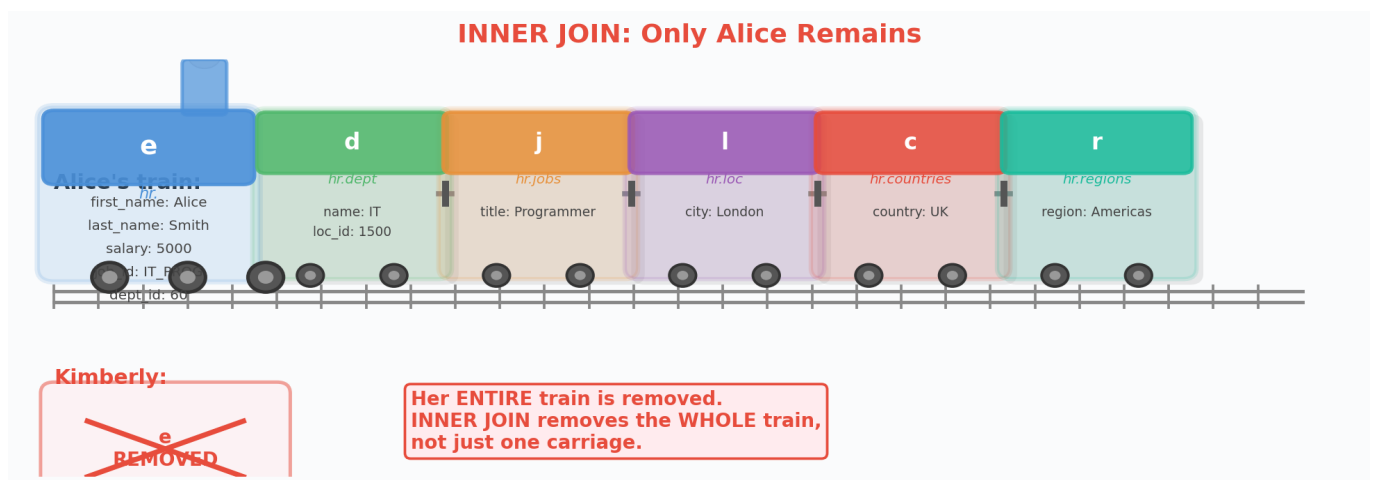


Figure 7: INNER JOIN = if coupling fails, remove the WHOLE train. Kimberly is gone.

Summary: The Train Rules

Rule 1: FROM = Locomotive

Every query starts with one table. That is your locomotive. The whole train is just this one car at the beginning.

Rule 2: JOIN = Attach a Carriage

Each LEFT JOIN attaches a new carriage to the train. The ON clause is the coupling - it defines how the new car connects.

Rule 3: One Train, Not Many

After connecting, there is only ONE train. Not "the e train" and "the d train". The aliases (e, d, j...) are just labels on the carriages of this single train.

Rule 4: Read from the Train, Not the Station

When you write "d.location_id" in a later JOIN, you walk to the "d" carriage of YOUR train and read location_id from there. You do NOT go to the station (the original departments table). That data has been in your train since Step 2.

Rule 5: LEFT JOIN = Keep Empty Carriages

If a coupling cannot be made (no matching data), LEFT JOIN still attaches the carriage but leaves it empty (NULL). The train stays intact.

Rule 6: INNER JOIN = Cut the Train

If a coupling cannot be made, INNER JOIN removes the ENTIRE train. Not just the empty carriage - everything. The whole row disappears.

LEFT JOIN = "Attach the carriage, leave it empty if no match"
INNER JOIN = "If the coupling fails, destroy the whole train"